TEC | Tecnológico de Costa Rica

# Proyecto 2

Estructuras de Datos

**S**antiago Eloy Céspedes Mora
**J**osé Pablo Barrantes Hernández
**J**orge Ulate Mora

**2**4/06/2026

# 1. Descripción del problema

El presente proyecto aborda la complejidad inherente a la gestión de redes interconectadas a gran escala, utilizando como analogía una red galáctica compuesta por estrellas, planetas y nebulosas. La inmensidad de este sistema, con sus billones de conexiones potenciales, presenta un desafío técnico significativo para la navegación y el transporte eficiente. La modelación de este entorno requiere la implementación de una estructura dinámica robusta que permita representar grafos orientados y no orientados con alta precisión.

El problema central radica en la necesidad de desarrollar un sistema computacional capaz de procesar estas redes bajo condiciones de alta demanda de datos. Los desafíos incluyen el cálculo de rutas óptimas mediante algoritmos de caminos mínimos, la gestión integral de naves espaciales, y el almacenamiento persistente de la información. Asimismo, el sistema debe ser capaz de consumir datos desde una API externa mediante protocolos HTTP, garantizando la integridad de la información y evitando redundancias. La complejidad se incrementa al requerir la implementación de un algoritmo de Kruskal modificado, el cual debe ser ejecutado de forma aleatoria sin generar ciclos, demandando un control riguroso sobre las estructuras de datos en memoria para evitar fallos críticos en el sistema.

# 2. Justificación

La optimización del tiempo de desplazamiento en entornos complejos constituye una necesidad crítica para cualquier sistema de transporte intergaláctico. Este proyecto se justifica por la importancia de implementar algoritmos de rutas mínimas y árboles de expansión, los cuales permiten minimizar los costos operativos y temporales en redes masivas. La aplicación de estos algoritmos no solo optimiza la eficiencia de los viajes, sino que demuestra la superioridad de las estructuras dinámicas frente a las estáticas en términos de flexibilidad y escalabilidad.

Además, el desarrollo del proyecto permite profundizar en el uso avanzado de punteros y *structs* en C++, aplicando un análisis crítico para la resolución de problemas de conectividad. La implementación de un sistema de gestión de archivos permite la persistencia de datos y la generación de reportes detallados, los cuales son esenciales para la toma de decisiones basada en datos. Finalmente, este proyecto cumple con el objetivo de integrar el conocimiento académico con herramientas de software reales, como *curl*, fortaleciendo las competencias técnicas necesarias para enfrentar desafíos de ingeniería de software a gran escala.

# 3. Solución propuesta

## 3.1. Ambiente de desarrollo

El desarrollo del sistema se llevó a cabo utilizando las siguientes herramientas de software, seleccionadas por su robustez y eficiencia:

- **C++:** Lenguaje de programación de propósito general, orientado a objetos, que destaca por su alto rendimiento y control preciso sobre la gestión de memoria. Su capacidad para manejar punteros dinámicos resulta esencial para implementar estructuras de datos complejas como grafos, garantizando la optimización de recursos.
- **Visual Studio:** Entorno de Desarrollo Integrado (IDE) de Microsoft, utilizado para la compilación y depuración del sistema. Este entorno facilita la gestión de librerías externas necesarias para el consumo de servicios web mediante *curl*, proporcionando herramientas avanzadas para asegurar la estabilidad del código.

## 3.2. Estructuras de datos utilizadas

Para la modelación lógica de la red galáctica y las entidades asociadas, se definieron estructuras (*struct*) personalizadas que permiten la persistencia en memoria y la vinculación eficiente entre nodos y registros.

| **Estructura** | **Descripción** |
|---|---|
| `Vertice` | Representa un nodo galaxia; contiene el identificador, punteros a la sub-lista de arcos y al siguiente nodo del grafo. |
| `Arco` | Define las conexiones (aristas) entre galaxias, incluyendo el costo del trayecto. |
| `Ship` | Entidad que agrupa la información de las naves, incluyendo su historial de viajes y estado operativo. |
| `Trip` | Almacena el registro detallado de un viaje específico realizado por una nave. |
| `Galaxy` | Estructura para almacenar atributos espaciales (coordenadas X, Y, Z) y datos de identificación. |
| `Edge` | Definición técnica de las rutas (aristas), incluyendo tiempo de duración y estado de la conexión. |
| `DijkstraResult` | Estructura auxiliar diseñada para el cálculo y almacenamiento de rutas óptimas entre galaxias. |

## 3.3. Funciones principales

El sistema se organiza en funciones modulares encargadas de la gestión de entidades y la lógica algorítmica:

- **Gestión CRUD:** Funciones dedicadas a la creación, lectura, actualización y eliminación de galaxias, naves y rutas.
- **Algoritmo de Kruskal:** Función especializada que implementa la variante aleatoria para la generación de la red galáctica de bajo costo.
- **Consumo de API:** Funciones que utilizan la librería *curl* para extraer datos en formato JSON desde el servicio *Mock API*.
- **Reportes:** Funciones encargadas de la escritura de archivos de texto con los resultados de las rutas optimizadas y estadísticas de conectividad.

## 3.4. Manejo de excepciones y utilidad técnica

Para asegurar la estabilidad del sistema, se implementaron bloques *try-catch* en las operaciones críticas, tales como la lectura de archivos, la conversión de tipos de datos JSON y la conexión con la API. El uso de *try-catch* es vital para capturar excepciones en tiempo de ejecución, permitiendo al sistema recuperar su estado o informar al usuario sobre el error sin colapsar.

## 3.5. Módulo de utilidades (Utils.cpp)

El archivo `utils.cpp` centraliza las funciones de soporte necesarias para la operatividad del sistema, garantizando un código modular y de fácil mantenimiento:

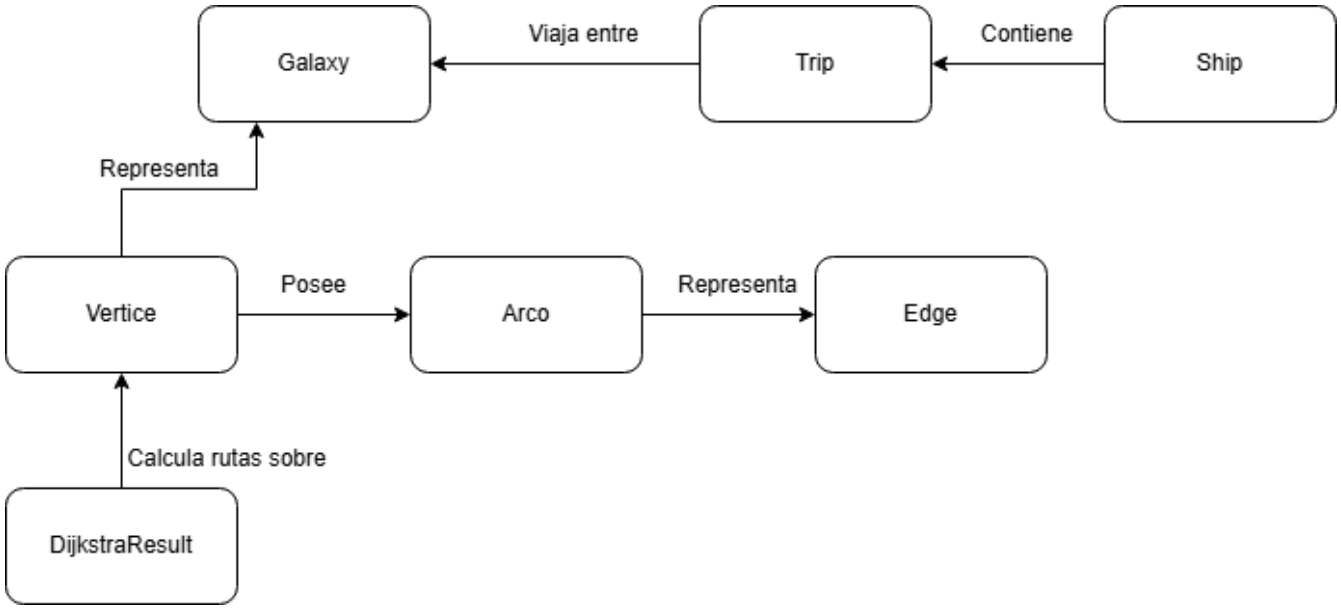
- **Validación robusta:** Implementa funciones para la captura de datos (enteros, flotantes, cadenas) que incluyen mecanismos de limpieza de *buffer*, evitando fallos ante entradas de usuario inesperadas.
- **Gestión multimedia:** Se integró la biblioteca *Windows Multimedia API* (`mmsystem.h`) para reproducir efectos sonoros mediante la función `PlaySound`. Esta implementación se configuró de manera asíncrona (*SND_ASYNC*), permitiendo una experiencia de usuario fluida sin interrumpir el flujo del programa.
- **Gestión de memoria:** Contiene la función `freeAll()`, esencial para el ciclo de vida del software, encargada de liberar todos los nodos dinámicos y estructuras de datos creadas durante la ejecución.

## 3.6. Implementación de canción

Listing 1: Implementación de utilidades del sistema.

```
// Reproduccion asincrona de archivos .wav
void playAudio(const char* filename) {
    // TEXT convierte el nombre del archivo al formato requerido
    // SND_ASYNC permite que el programa continue ejecutandose
    PlaySound(TEXT(filename), NULL, SND_FILENAME | SND_ASYNC);
}
```

## 3.7. Diagrama de estructuras



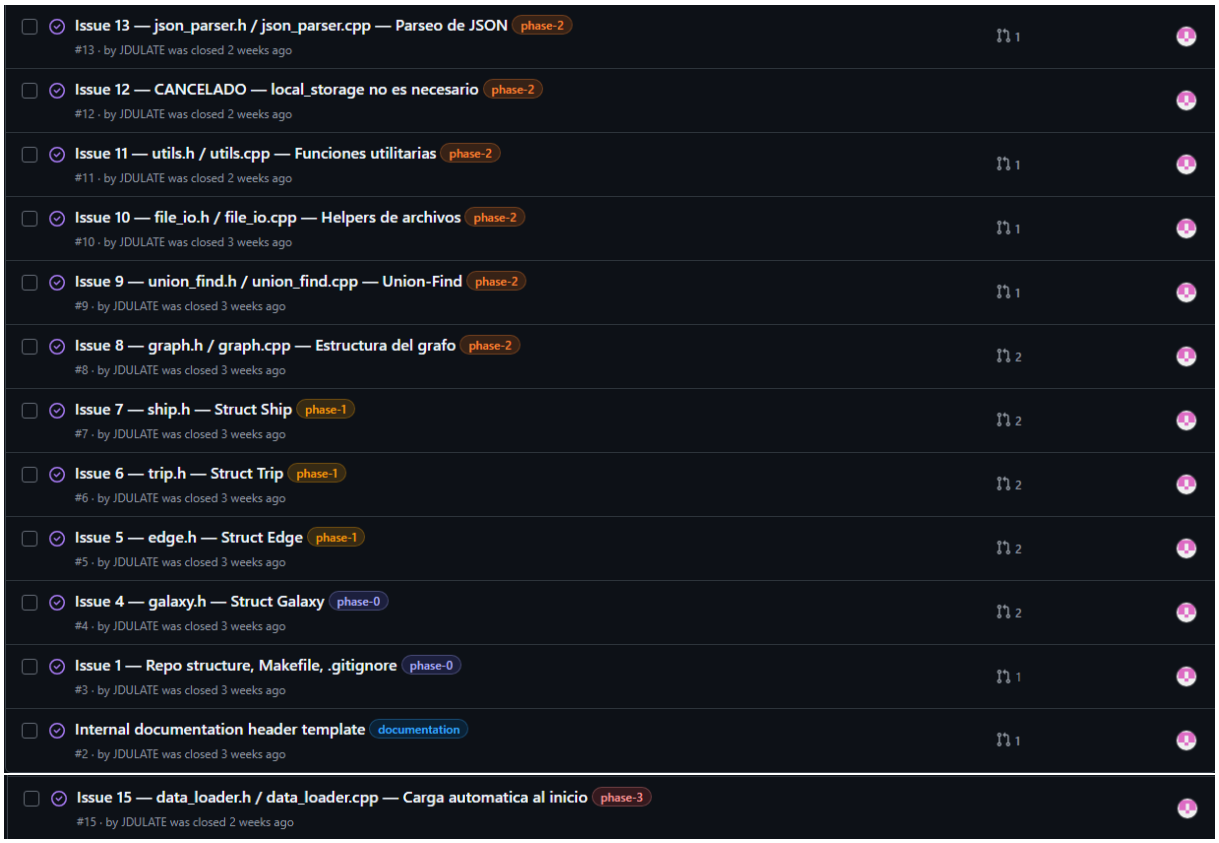

## 3.8. Minuta de trabajo Santiago



## 3.9. Minuta de trabajo José Pablo




## 3.10. Minuta de trabajo Jorge



# 4. Análisis de resultados

A continuación, se presenta un análisis del cumplimiento de los requerimientos planteados en el proyecto, evaluando las funcionalidades implementadas y su estado final.

| Requerimiento | Estado | Observaciones |
|---|---|---|
| **Estructuras de Datos** | | |
| Grafo mediante lista de adyacencia (Vértices y Arcos) | Completo | Representación dinámica del mapa galáctico mediante listas enlazadas. |
| Galaxias | Completo | Almacena identificador, nombre y coordenadas espaciales. |
| Rutas (Edge) | Completo | Contiene información del costo, duración y estado de las conexiones. |
| X-Wings (Ship) | Completo | Registro de naves con historial de viajes asociado. |
| Viajes (Trip) | Completo | Almacena los recorridos realizados entre galaxias. |
| Resultado de Dijkstra | Completo | Estructura auxiliar para almacenar caminos mínimos calculados. |
| **Operaciones CRUD** | | |
| CRUD de galaxias | Completo | Permite agregar, modificar, eliminar y consultar galaxias. |
| CRUD de rutas | Completo | Administración completa de conexiones entre galaxias. |
| CRUD de naves | Completo | Gestión de la información de las naves registradas. |
| CRUD de viajes | Completo | Registro y mantenimiento del historial de viajes. |
| Validación de datos duplicados | Completo | Se evita el registro de identificadores repetidos. |

| Requerimiento | Estado | Observaciones |
|---|---|---|
| **Algoritmos** | | |
| Algoritmo de Dijkstra | Completo | Determina la ruta de menor costo entre dos galaxias. |
| Algoritmo de Kruskal Modificado | Completo | Construcción del árbol de expansión utilizando aristas aleatorias proporcionadas por la API. |
| Recorrido DFS | Completo | Obtiene todas las rutas posibles desde una galaxia origen. |
| Union-Find | Completo | Utilizado para detectar ciclos durante la ejecución de Kruskal. |
| **Consultas** | | |
| Mostrar todas las rutas posibles | Completo | Implementado mediante recorrido en profundidad (DFS). |
| Ruta de menor costo | Completo | Consulta basada en el algoritmo de Dijkstra. |
| Árbol de expansión mínima | Completo | Presenta el resultado obtenido por Kruskal. |
| Historial de viajes por nave | Completo | Consulta de todos los viajes registrados para una nave. |
| **Reportes** | | |
| Reporte de rutas más cortas | Completo | Generación automática del archivo de resultados de Dijkstra. |
| Reporte del árbol optimizado | Completo | Exportación del árbol de expansión mínima generado por Kruskal. |
| Reporte de estadísticas | Completo | Incluye información sobre rutas calculadas y conectividad del grafo. |

| Requerimiento | Estado | Observaciones |
|---|---|---|
| Almacenamiento en archivos txt | Completo | Los resultados son persistidos en archivos de salida. |
| **Carga y Persistencia de Datos** | | |
| Consumo de API REST | Completo | Obtención automática de galaxias y rutas mediante servicios web. |
| Procesamiento de JSON | Completo | Interpretación y carga de la información recibida desde la API. |
| Carga automática del grafo | Completo | Construcción dinámica del grafo a partir de los datos obtenidos. |
| Lectura y escritura de archivos | Completo | Manejo de archivos de configuración y reportes del sistema. |
| **Resultados Generales** | | |
| Representación correcta del grafo | Completo | Implementación mediante estructuras dinámicas y listas de adyacencia. |
| Organización modular del sistema | Completo | Código dividido por módulos, entidades, algoritmos y utilidades. |
| Uso de estructuras dinámicas | Completo | Implementación basada en punteros sin utilizar contenedores STL para la estructura principal. |
| Cumplimiento de los objetivos del proyecto | Completo | El sistema satisface los requerimientos establecidos en el enunciado del proyecto. |

# 5. Conclusiones

- **Gestión de Memoria y Estructuras Dinámicas:** La implementación basada exclusivamente en el uso de punteros y estructuras personalizadas permitió un control eficiente de los recursos del sistema. También permitiendo comprender como funcionan este tipo de estructuras en profundidad y su gestión en C++.
- **Eficacia Algorítmica:** Se logró integrar con éxito algoritmos de optimización complejos, tales como el Kruskal modificado y Dijkstra. La correcta implementación de estructuras auxiliares, como la técnica Union-Find, fue determinante para garantizar la integridad de los resultados y la eficiencia en la búsqueda de rutas mínimas en redes de gran escala, demostrando la aplicabilidad de los grafos en problemas de conectividad real.
- **Análisis Crítico y Resolución de Problemas:** El proyecto permitió desarrollar un pensamiento analítico frente a la validación de integridad de datos. La implementación de filtros para evitar registros duplicados, el manejo de excepciones y la persistencia de datos en archivos txt aseguran que el programa mantenga un estado consistente, cumpliendo satisfactoriamente con todos los objetivos que se solicitaron.

# 6. Recomendaciones

- **Implementación de Interfaz Gráfica (GUI):** Dado que el sistema actual opera mediante una interfaz de consola, se recomienda para futuras etapas integrar una interfaz gráfica funcional. Una GUI permitiría la visualización dinámica del mapa galáctico, donde el usuario pueda observar gráficamente las galaxias y las rutas, facilitando la interpretación visual de los resultados de los algoritmos de Kruskal y Dijkstra, mejorando así la experiencia de usuario y la accesibilidad del sistema.
- **Persistencia en Base de Datos Relacional:** Se sugiere migrar el modelo actual de archivos de texto (.txt) hacia un sistema de gestión de bases de datos relacional. Esto permitiría realizar consultas complejas mediante SQL, garantizando una mayor integridad referencial, transacciones seguras y una gestión mucho más eficiente de volúmenes de datos masivos, superando las limitaciones de lectura/escritura secuencial de los archivos planos.
- **Optimización de Algoritmos para Alta Densidad:** Para escenarios de galaxias con una densidad de conexiones extremadamente alta, se recomienda sustituir o complementar las estructuras actuales con estructuras de datos de mayor rendimiento. Esto permitiría reducir significativamente la complejidad temporal en la

búsqueda de rutas óptimas, manteniendo un tiempo de respuesta constante incluso cuando el número de aristas crezca exponencialmente.

# 7. Bibliografía

# Referencias

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*, 269–271. https://doi.org/10.1007/BF01386390

[2] El Universo. (2019, abril 12). *1.560 millones de litros de bebidas se toman al año*. Recuperado de `https://www.eluniverso.com/noticias/2019/04/12/nota/7281038/1560-millones-litros-bebidas-se-toman-ano`

[3] ISO/IEC. (2020). *Programming languages — C++* (Estándar No. ISO/IEC 14882:2020). https://isocpp.org/std/the-standard

[4] Stenberg, D. (2026). *libcurl: The multiprotocol file transfer library*. https://curl.se/libcurl/

[5] Tecnológico de Costa Rica. (2026). *Proyecto programado: Grafos y Archivos*. Centro Académico de Alajuela. https://galaxias-mock-api.onrender.com/